Sejuk Sejuk Service — Operations System

A simplified internal operations system for a fictional aircon service company, built for the Programmer Assessment (Operations System + AI Challenge). Covers the full workflow:

Order → Assignment → Service Completion → Manager/Accounts Review, plus an AI assistant that answers operational questions from live system data.

What Was Built

All modules from the assessment were implemented:

- **Module 1 — Admin Portal**: create orders, assign a technician (with a "Notify Technician via WhatsApp" button on assignment), edit/reschedule/cancel orders, and browse/search/filter/sort/export all orders.
- **Module 2 — Technician Portal**: job list (manual refresh, a direct **Call Customer** button, completed jobs shown dimmed), start job, complete job with up to 6 file uploads, auto-calculated final amount, a per-service-type **checklist** (different standard tasks for cleaning/repair/gas refill/installation/inspection) that ticks straight into the Work Done field, a live elapsed-time counter from "Start Job" to submission (persisted through a page reload via `localStorage`, shown in the confirm dialog and the completed screen), and optional payment capture (amount, method, and receipt photo — the full bonus field set). The completion submit is retry-safe: if a connection drops mid-upload out in the field and the technician taps submit again, it reuses the existing completion record instead of creating a duplicate, and shows which step ("Uploading photo 2 of 4…") is in progress.
- **Module 3 — WhatsApp Notification Trigger**: on job completion, a "Notify Customer" button opens a `wa.me` deep link pre-filled with the completion message. The same `wa.me` pattern is reused on the Admin side to notify the assigned technician when an order is created or (re)assigned to them, and on the Technician side (Module 2 bonus: "Notify manager/accounts when job completed") to notify each manager who has a phone number on file — that message includes the same over-quote/no-photos flags as the AI Workflow Supervisor, so a manager sees on WhatsApp itself whether a job needs a closer look before opening the app.
- **Manager Review Queue**: review `Job Done` orders and close them, with inline **AI Workflow Supervisor** flags (final amount much higher than quoted; job done with no photos uploaded).
- **Real authentication (bonus)**: Supabase Auth (email/password) instead of a mock role switch, with role-scoped Row Level Security enforced in Postgres, not just the UI.
- **Bonus — KPI Dashboard**: jobs completed, total amount, and a leaderboard/chart per technician (including a Postpone/Reschedule count), with a 7/30-day range toggle in the Manager view — which is also the Manager's post-login landing page, and leads with "Awaiting Review" and "Flagged" counts (jobs matching the AI Workflow Supervisor rules) linking straight to the Review Queue, so anything needing attention is visible before drilling into performance stats. The Admin view has its own operational dashboard: order-status breakdown, technician workload, revenue trend, and upcoming (next 7 days) schedule.
- **AI Module — Operations Query Window**: a chat-style assistant in the Manager view that answers the three example question types using controlled, parameterized Supabase queries — never a raw/unrestricted database query — with DeepSeek used only to phrase the final sentence from the retrieved rows.
- **AI Operational Insight (optional advanced challenge)**: a fourth query type — "Which technician might be overloaded (this week)?" — compares each technician's completed-job count against the team average for the period and flags anyone significantly above it. Same controlled-query pattern as the rest of the AI Module: the comparison is computed in plain code from a parameterized Supabase query, and DeepSeek only phrases the result.
- **AI Document Understanding (optional advanced challenge)**: on the New Order page, Admin can upload a photo of a service request/quote/invoice; it's OCR'd in the browser (Tesseract.js), the extracted text is sent to DeepSeek to pull out customer name, phone, address, service type, service details, and amount, and the

form pre-fills from that — Admin still reviews and can edit every field before submitting, the same "AI assists, human confirms" rule as the rest of the app.

- **Beyond the assessment spec**: an Admin Calendar (month view, color-coded by technician) and Schedule (day-grouped upcoming jobs) for visualizing `scheduled_at`; order cancellation (only while `New` / `Assigned`, with a required reason) and reschedule tracking (changing an already-scheduled date, with an optional reason); a filterable, paginated Audit Log viewer; and, on the Technician side, a **Dashboard** (now the post-login landing page) with a weekly-progress ring, this-month/today stats with trend-vs-last-month indicators, a weekly completions chart, a service-type breakdown chart, and next-job/recent-completions views, plus a **History** page listing every completed job with expandable full details (work done, amounts, payment, attachment thumbnails, receipt photo). All three roles now share the same sidebar navigation shell (`DashboardLayout` / `AppSidebar`) rather than Technician having a separate, simpler mobile header.
- **In-app notification system (beyond the assessment spec)**: a bell icon in the dashboard header, shared by all three roles, backed by a `notifications` table (`supabase/schema.sql`) and Supabase Realtime — a new notification appears live, without a page refresh, via a `postgres_changes` subscription scoped to the signed-in user (`src/hooks/useNotifications.ts`). Complements the WhatsApp notifications (Module 3), which are for the *customer*; this is the in-app channel for *staff*. Wired into every workflow step: Admin assigning/reassigning/rescheduling/cancelling a job notifies the technician; a technician completing a job notifies every Manager; a Manager reviewing a job notifies that technician back, and closing it notifies every Admin (`src/lib/notifications.ts`, called from the same places `logAction()` already is). Clicking a notification marks it read and navigates straight to the relevant page.

Every status change, technician assignment, cancellation, and reschedule is written to the `audit_log` table for traceability, per the assessment's basic system rules. Reschedules are additionally recorded in `order_reschedules`, which is what feeds the KPI Dashboard's Postpone/Reschedule column.

Tech Stack

Layer	Tool
Front-end	React 19 + TypeScript + Vite
Styling	Tailwind CSS v4
Routing	React Router
Backend/DB	Supabase (Postgres + Row Level Security)
File storage	Supabase Storage (`job-attachments` bucket)
AI	DeepSeek (`deepseek-chat`, OpenAI-compatible API)
OCR	Tesseract.js (client-side, for AI Document Understanding)
Serverless functions	Vercel Node functions (`api/ai-query.ts`, `api/ai-extract-document.ts`)
Auth	Supabase Auth (email/password) — real login, role-scoped RLS
Deployment	Vercel

Architecture Decisions

- **Real authentication with role-scoped RLS**: the assessment explicitly says a mock role switch is *sufficient*, but real Supabase Auth was implemented as the documented bonus. `profiles ( user_id, role, name )` extends `auth.users` with app-specific data; `technicians.user_id` links each technician row to their own account. `AuthContext` tracks the real Supabase session via `onAuthStateChange` and loads `role` / `name` / (for technicians) their linked `technicianId` once signed in — `RequireRole` gates routes against that instead of a `localStorage` value.
- **Access control lives in Postgres RLS, not just the UI.** Two SQL helper functions, `auth_role()` and `auth_technician_id()` , are called from every policy: admins/managers see and manage everything; a technician can only see/update orders (and insert completions/attachments for orders) assigned to them. This means even a compromised or buggy client can't read or write data outside what the signed-in user is allowed — the check isn't just "does the button render," it's enforced at the database.
- **No self-service role assignment.** `profiles` has a `SELECT` policy (read your own row) and deliberately no `INSERT` / `UPDATE` / `DELETE` policy for regular users — the only way to create or change a profile is a service-role action (dashboard or `scripts/seedUsers.mjs`), so no logged-in user can ever grant themselves Admin.
- **Manager contact lookup is narrowly scoped, not a general profiles read.** The "Notify Manager via WhatsApp" button (Module 2 bonus) needs a technician to read the manager's name/phone from `profiles` , but the existing policy only lets a user read *their own* row. Rather than opening `profiles` up broadly, there's a second, additive `SELECT` policy scoped to `role = 'manager'` only — any signed-in user can see manager contact rows (low-sensitivity, needed for this feature), but admin and other technicians' rows stay private. **If you're on an existing database, re-run `schema.sql`** to pick up this policy and the new `profiles.phone` column.
- **Status transitions as a small pure helper, not DB constraints** (`src/lib/orderStatus.ts`): `New → Assigned → In Progress → Job Done → Reviewed → Closed` . The *sequence* is enforced in the UI/mutation layer rather than a Postgres state machine; RLS enforces *who* can update a given order, not which status values are reachable from which.
- **AI query classification is deterministic, not LLM-driven** (`server/aiQueryCore.ts`): the question is matched against a small set of regex patterns to pick one of three supported query shapes and a date range, which are then run as ordinary parameterized Supabase queries. DeepSeek is only called *after* the data is retrieved, purely to phrase the final sentence — it never decides what to query or sees the whole database. If the DeepSeek call fails or no key is configured, the endpoint falls back to a template-formatted answer built directly from the retrieved rows, so the feature degrades gracefully instead of breaking.
- **Job completion is upsert-like, not insert-only**: `JobComplete.tsx` looks for an existing `service_completions` row for the order before writing one, so a technician re-submitting after a dropped connection mid-upload updates that row instead of creating a second completion for the same order (which would otherwise silently confuse the Manager Review Queue, which only reads the first one). This needed a new `service_completions` **update** RLS policy in `schema.sql` — the table previously only had `select` / `insert` policies, so technicians could create a completion but not correct it. **If you're on an existing database, re-run `schema.sql`** to pick this up.
- **`service_completions.started_at`** records when a technician tapped "Start Job" (tracked client-side via `localStorage` so it survives a page reload), written on completion submit when available, and drives the per-job duration shown on the Technician Dashboard and History pages. This column already existed on the live database before `schema.sql` tracked it — added here for reproducibility on a fresh install. Existing databases already have it; no action needed there.
- **Document understanding is OCR-then-LLM, not vision-model-then-LLM.** DeepSeek's public API is text-only — it cannot inspect image pixels — so the document photo never leaves the browser as an image: Tesseract.js OCRs it client-side into plain text first, and only that text is sent to the server/DeepSeek. This keeps the same "AI only sees data that was already retrieved through a controlled step" rule the AI Query module uses, just with OCR as the retrieval step instead of a Supabase query. Unlike AI Query, there's no

template fallback if DeepSeek is unavailable — there's no sensible non-AI way to turn raw OCR text into structured fields, so a missing key or failed call surfaces as an explicit error instead of a degraded answer.
- **Server code lives outside `src/`** (`server/` , `api/`): `src/` is compiled under a browser-oriented `tsconfig` (DOM lib, no Node types). AI-query logic needs `process.env` and runs server-side only, so it's a sibling top-level folder with its own `tsconfig.server.json` , imported by both `api/ai-query.ts` (the real Vercel function) and a small Vite dev-server middleware (`vite.config.ts`) that reuses the exact same function during `npm run dev` , so the AI module is testable locally without `vercel dev` .

AI Query Types Supported

1. "How many jobs were completed **today / this week / last week**?"
2. "Which technician completed the **most jobs** (this week / last week)?"
3. "Which technician **might be overloaded** (this week / last week)?" — flags a technician when 2+ technicians were active in the period, they completed at least 3 jobs, and their count is more than 1.4× the team average for that period. Both numbers are an arbitrary but documented heuristic, not a business rule from the assessment — tuned for legibility on small seed datasets, not calibrated against real volume.
4. "What jobs did **technician <name>** complete (today / this week / last week)?" (name must be one of the seeded technicians: Ali, John, Bala, Yusoff)

If a question doesn't match any of these four shapes, the assistant returns a fixed message explaining what it can answer and suggests a rephrase — it does not attempt a best-effort guess against the database.

`api/ai-query.ts` requires a real signed-in session: the client sends the caller's Supabase access token, the endpoint validates it and checks their `profiles.role` is `manager` (401 if not signed in, 403 if signed in but not a manager) *before* running any query — every underlying Supabase call then runs as that authenticated user, so RLS scopes the data the same way it would for any other request they made.

"This week" / "last week" are implemented as rolling 7-day windows (last 7 days / the 7 days before that), not calendar weeks — simpler and avoids timezone-boundary edge cases, but won't match a strict Monday-start week if that's expected.

AI Document Understanding — What's Supported

Upload a photo (not a PDF — see Limitations) of a service document on the New Order page; it extracts `customer_name` , `phone` , `address` , `service_type` , `service_details` , and `amount` , and pre-fills the matching form fields (the extracted "date" is returned by the API but not auto-filled anywhere, since New Order has no date field to put it in — `created_at` is automatic). `phone` / `address` go beyond the assessment's example field list, added because the New Order form actually needs them and they were sitting right there in the OCR text unused. Any field DeepSeek can't find in the OCR text comes back `null` and that form field is left as-is rather than being overwritten with a guess.

OCR runs with both English and Malay language data loaded (Tesseract.js `createWorker(['eng', 'msa'])`), since this is a Malaysian business and real documents are likely to mix both. If the extracted `service_type` doesn't fuzzy-match any of the fixed dropdown options, the dropdown is left unchanged **and a visible amber notice names the unmatched value** — earlier versions of this feature would silently leave the field untouched with no indication anything had failed, which risked an admin thinking it was correctly extracted when it wasn't touched at all. A "Show raw text read from the document" toggle displays the exact OCR output, so a bad extraction (garbled photo vs. a DeepSeek miss on otherwise-clean text) can be diagnosed instead of guessed at. Uploads over 8MB are rejected up front with a clear message, rather than letting OCR hang on a large phone-camera photo with only a generic spinner.

`api/ai-extract-document.ts` requires a real signed-in session with `profiles.role = 'admin'` (401/403 otherwise), same pattern as the AI Query endpoint.

Limitations

- No public sign-up — accounts are provisioned up front (dashboard or `scripts/seedUsers.mjs`), which is intentional for an internal tool with a fixed, known set of staff, but means adding a new technician later is a manual step, not a self-serve flow.
- Row-level status *sequencing* (e.g. a technician can't skip straight to `Closed`) is still enforced only in the UI/mutation layer, not a Postgres state machine — RLS enforces *who* can write to a row, not which status transitions are valid.
- AI query classification only recognizes the four documented question shapes; it does not handle multi-part questions, ambiguous phrasing, or technicians outside the seeded list.
- "Postpone / Reschedule" is tracked (`order_reschedules`, populated when Admin changes an already-*scheduled* order's date) and shown per technician on the KPI Dashboard, but only reschedules of orders that already had a `scheduled_at` count — setting the first schedule on a previously-unscheduled order is treated as a normal edit, not a reschedule, since there's nothing to postpone yet.
- Row-selection checkboxes on the Orders list support one bulk action: assigning a technician to every selected order at once (bumping status `New → Assigned` for previously-unassigned ones, same rule as the single-order edit form). Selection is page-scoped (selecting "all" selects the current page, not every filtered row) and there's no bulk status change, cancel, or reassignment-with-notification-suppressed — just the one action. CSV export doesn't depend on selection either way; it exports every row matching the current filters.
- The `job-attachments` Storage bucket is marked "public" at the bucket level (so `<img>`/download links work directly) and read/write through the authenticated storage API is now scoped the same way as the rest of the app — admin/manager see everything, a technician only their own assigned orders' files (via the `${order_id}/…` path prefix each upload uses). That said, Supabase serves public-bucket objects through `/object/public/…` (what `getPublicUrl` returns) **without evaluating RLS at all**, so anyone with a leaked or guessed object URL can still fetch it — the RLS scoping only tightens the authenticated API surface, not the public URL itself. Closing that fully would mean a private bucket + signed URLs everywhere the app calls `getPublicUrl`, which wasn't done here.
- **AI Document Understanding** only accepts image files (JPG/PNG/etc.), not PDF — Tesseract.js OCRs images directly; a PDF would need to be rendered to an image first, which wasn't built. Accuracy also depends entirely on photo quality/lighting/handwriting legibility, same as any OCR pipeline — a blurry or handwritten document will extract poorly or return mostly `null` fields. Tesseract.js also fetches its OCR engine/language data from a public CDN at runtime by default rather than being fully self-hosted, so this feature (unlike the rest of the app) needs outbound internet access beyond just Supabase/DeepSeek to work.
- No automated tests.
- **Notifications insert policy is scoped by order, not by exact recipient.** A signed-in user can insert a `notifications` row for any `user_id` as long as they themselves have visibility into the referenced `order_id` (admin/manager, or the technician assigned to it) — the same rule as `orders select`. This stops a user from notifying an arbitrary recipient about an order they have no access to, but within an order they're allowed to see, Postgres still doesn't verify the recipient is the "correct" one (e.g. a technician could in principle address a notification to another technician instead of a manager) — the app's own call sites (`src/lib/notifications.ts`) are what actually constrain that, same trust model as `audit_log`.
- Notifications are in-app only — no push, email, or SMS delivery, and nothing is sent if the recipient's browser tab isn't open (no service worker). The bell only reflects what's already in the `notifications` table when the page loads, plus whatever arrives over the live Realtime subscription afterward.
- An existing project must **re-run** `supabase/schema.sql` to pick up the `notifications` table, its RLS policies, the "read admin profiles" policy, and the tightened `notifications insert` / `job-attachments` storage policies described above — it's additive and idempotent (safe to run again in full), but it doesn't run itself.

Local Setup

1. `npm install`
2. Create a Supabase project, then run `supabase/schema.sql` in its SQL Editor (creates tables, RLS policies/helper functions, seeds technicians, creates the `job-attachments` storage bucket).
3. Copy `.env.local.example` to `.env.local` and fill in:
   - `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` — from Supabase project settings
   - `DEEPSEEK_API_KEY` — from platform.deepseek.com (optional; without it, the AI module falls back to template-formatted answers instead of DeepSeek-phrased ones)
4. Provision the real accounts (Admin, Manager, and one per technician):
   - Easiest: get your **service_role** key from Project Settings → API, add it to `.env.local` as `SUPABASE_SERVICE_ROLE_KEY` (never `VITE_`-prefixed — it must never reach the browser), then run `node scripts/seedUsers.mjs`. It creates all 6 accounts with random passwords (printed once to the console — save them), their `profiles` rows, and links each technician to their account. Safe to re-run.
   - Or manually: create the 6 users in Authentication → Users, then insert matching `profiles` rows and set `technicians.user_id` yourself via the Table Editor.
5. `npm run dev` — the AI query endpoint works locally too, via a Vite dev middleware that mirrors `api/ai-query.ts`.

Deployment

Push to a Git repo and import into Vercel, or run `vercel`. Set the same three env vars in the Vercel project's Environment Variables settings. `api/ai-query.ts` is auto-detected as a serverless function; `vercel.json` rewrites all non-`/api` paths to `index.html` for client-side routing.

Self-Assessment

- **Easiest module**: Module 1 (Admin Portal) — a standard form-to-database flow with no unusual constraints.
- **Hardest part**: keeping the AI module's data access genuinely constrained rather than just prompting an LLM to "be careful" — the design that made this straightforward was moving query *selection* out of the LLM entirely (regex classification → parameterized query → LLM only formats already-retrieved rows).
- **What I'd improve for production**: a proper state machine enforced in Postgres (or at least a DB trigger) for status *sequencing*, instead of only in the client — RLS now covers *who* can write, but not *which transition*; per-attachment storage scoping instead of "any signed-in user"; a self-serve (but admin-approved) way to add technicians instead of a manual seed script; and replacing the regex-based AI query classifier with a small tool-calling setup (LLM picks from a fixed set of typed query functions) so more question phrasings are understood without expanding the regex list by hand.
- **AI tool usage while building**: built interactively with Claude Code, which wrote the schema, components, and serverless function from the assessment spec.